

CS 122A Custom Laboratory Project Report

Blackjack Table

Timothy Flannery

6/8/2026

Introduction

The project is a multiplayer blackjack game system. The players each connect their personal device (phone, tablet, computer) to the Pico's Wi-Fi access point to open the hosted web server to interact with the game. This communication is event driven both ways. When the game is interacted with by the player that event triggers changes to the main game states, and events in the main game states trigger changes to the players webpage states.

The Pico communicates a lvgl generated screen via spi to be displayed by the FPGA controller on the LCD screen. The fpga reads spi, stores the frame in sdram, and sends it out to the LCD screen. The LCD screen functions as the main table/dealers table showing current game states, the dealers hand and card animations.

Elements of Complexity

- Real-time multiplayer web interface: The webpage uses Server-Sent Events (SSE) so each connected device receives live game-state updates from the Pico the instant something changes (cards dealt, turn changes, bets placed)
- Concurrent LCD rendering and networking: The LVGL UI, the game state machine, and the lwIP-based HTTP server all run cooperatively, so the display, animations, and network connections all stay responsive simultaneously
- Custom HTTP/SSE server built directly on lwIP: The server was written against the raw lwIP TCP API. It handles HTTP parsing, routing, and persistent SSE connections per player, including proper connection teardown and panic-free buffer management

- Graceful multiplayer connect/disconnect handling: Each player has their own SSE connection tied to a Player struct. When a player closes their browser or loses connection, the system detects it, marks them inactive, resets their hand/bet, and keeps the game running smoothly for the remaining players.
- FPGA as a dedicated display controller: The IceSugar-pro acts as a display framebuffer that utilizes the onboard SDRAM to display a simple LVGL application driven by the Raspberry Pi Pico. The framebuffer is communicated between the two devices through the SPI communication protocol.
- LVGL image rendering: Using images decoded into a C array of pixel data LVGL displays custom images stored directly on the Pico.
- LVGL animations: The card images are passed into a custom animation function that moves the card to the desired location in the allotted amount of time.
- Not implemented FreeRTOS: Tried implementing FreeRTOS into the system to run LVGL rendering, game logic, and network communication as separate tasks but was unable to implement.

User Guide

- 1) Power on the Pico 2 W and the FPGA board
- 2) Each player connects their phones Wifi to the Pico's Wifi access point named "Blackjack Table"
- 3) Once connected to the network each player opens their browser to <http://192.168.4.1> which will open the game webpage
- 4) At the start of each round one player must hit the start button to begin the round
- 5) Each round functions like a basic blackjack game. Players receive prompts, view their cards, and use the buttons on their device to place their bets, and choose to hit or stand during their turn
- 6) When done playing close the browser window and disconnect from the network

Hardware Components Used

- Raspberry Pi Pico 2 W
- Icesugar-Pro FPGA
- PMOD RGBLCD Expansion Board
- 480x272 RGB LCD display
- Personal devices for players (phone, tablet, computer)

Software Libraries Used

- `Pico_stdlib`: Basic pico library functions.
- `hardware_spi`: Provides the low-level driver for the Pico's SPI. Used to send pixel data to the fpga.
- `pico_cyw43_arch_lwip_threadsafe_background`: Used for WiFi and network implementation. It initializes the CYW43 WiFi chip on the Pico 2 W and integrates the lwIP TCP/IP stack so it runs safely in the background via interrupts/callbacks rather than having to manually poll it on every loop iteration. Lets lwIP's callbacks fire correctly while the main loop is busy doing other things.
- `dhcpserver`: When a player's device connects to the Pico's WiFi network, it automatically hands it an IP address so it can immediately reach the HTTP server without manual network configuration.
- `LVGL`: Builds and renders everything seen on the LCD: the card graphics, chip graphic, panels, status text, and animations. They are put into the frame buffer that gets sent out over SPI to the FPGA.
- `pico_rand`: Provides hardware-backed random number generation. Used for deck shuffling for randomized cards each round.

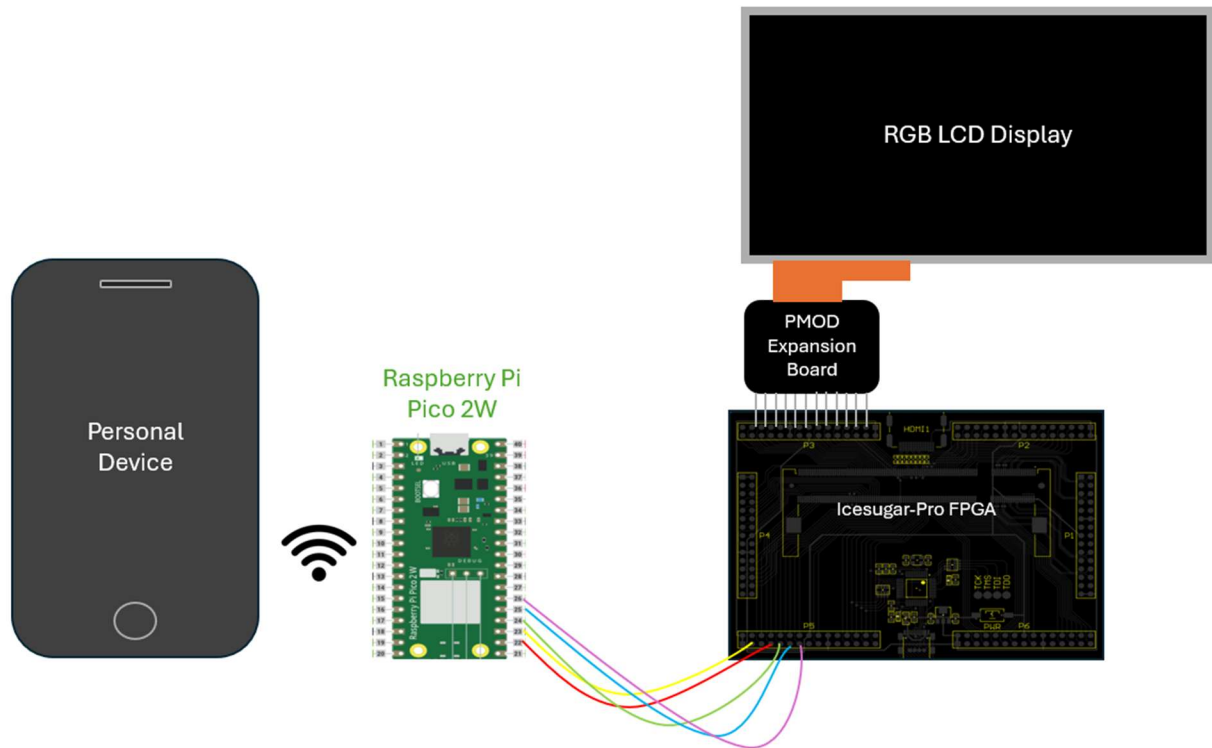
Protocols Used

- **SPI**: Used for the link between the Pico and the FPGA, carrying the rendered frame/pixel data from LVGL out to the display controller.
- **WiFi**: The wireless communication standard the CYW43 chip uses to broadcast the access point and exchange data with player devices over radio.
- **TCP**: Ensures reliable, ordered, and error-checked delivery of data between the pico and player devices.
- **IP**: Used for the addressing and routing of data so they reach the correct destination.
- **DHCP**: Defines how a device requests an IP address and how the server responds with one when a device joins your access point.
- **HTTP**: Used by the web server to serve the game webpage and handle player actions.

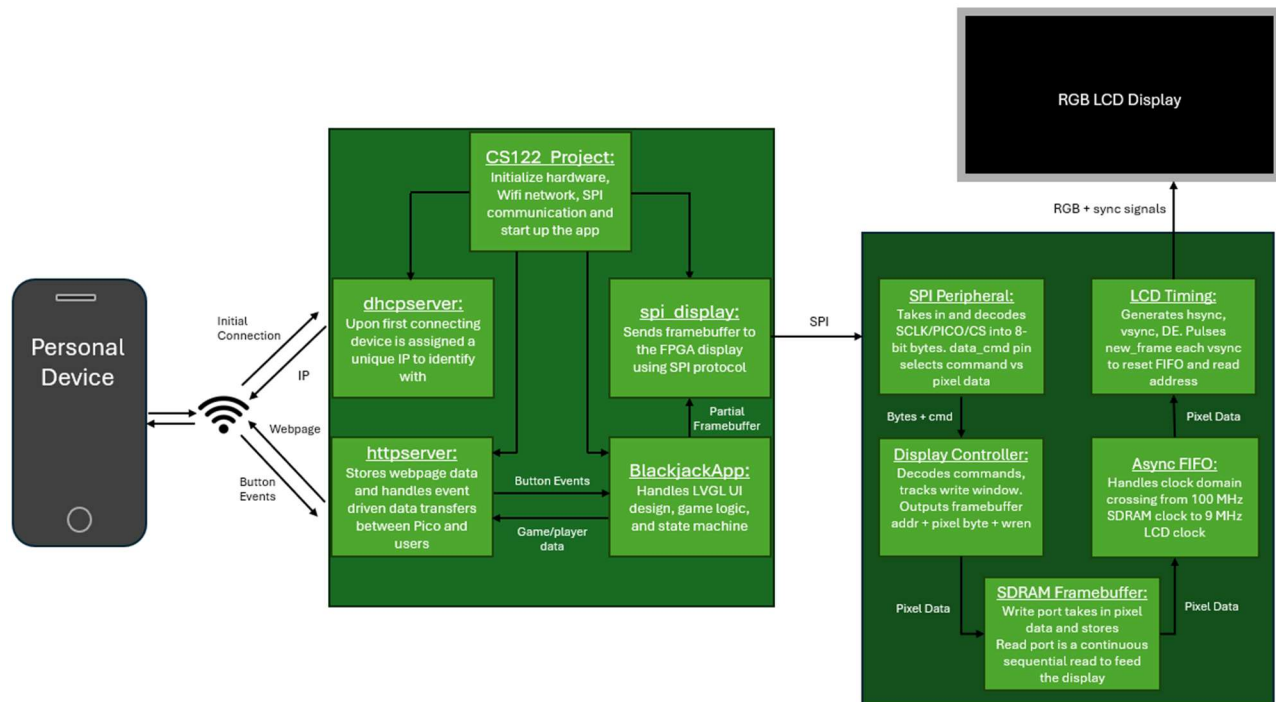
Requirements

- 1) Raspberry Pi Pico is the main brain of the games operation.
 - Hosts a dedicated Wifi connection point with a webserver for the players to interact with.
 - Runs lvgl and generates frames for the LCD display (Labels, objects, images, custom fonts)
 - Sends lcd frames to fpga via spi
 - Controls game and player states and handles game logic
- 2) Icesugar-pro FPGA: Served as the display driver for the LCDD display
 - Read spi signal from the Pico and stored the frame into sdram
 - Sent the frame to be displayed on the LCD screen through the PMOD RGBLCD Expansion Board
- 3) RGB LCD Display: displays the main table
 - Shows main game states
 - Displays dealers cards
 - Card animations
- 4) Event Driven Execution:
 - Networking is entirely callback-driven
 - Game-state pushes are triggered by state changes

Wiring Diagram



Design Diagram



AI Usage

- Initially tried using `pico_lwip_http` for the http server but kept running into errors caused by the library's restrictions. Used Claude to help guide in writing and generate parts of the custom `http_server` implementation
- I've never written HTML code before so AI aided in creating HTML code for the webpage
- Aided in understanding provided code from Allan to help combine my prior implementations with provided code
- General researching, answering questions, and planning aid

Acknowledgements

- Pico SDK documentation: [Raspberry Pi Documentation - Pico C SDK](#)
- Raspberry Pi pico-examples repository: [GitHub - raspberrypi/pico-examples · GitHub](#)
- LVGL: [LVGL — Light and Versatile Embedded UI Ecosystem](#)
- Allan Knight
- Claude/Gemini

